Sistemas Distribuidos I
(75.74)

Sistemas elásticos y de alta disponibilidad

Técnicas de Diseño. Elasticidad. Disponibilidad.

Docentes

- Pablo D. Roca
- Ezequiel Torres Feyuk

Agenda

- **Escalabilidad**
- Elasticidad
- Alta Disponibilidad

Escalabilidad | Objetivos

Quizá el objetivo más importante para los sistemas distribuidos modernos.

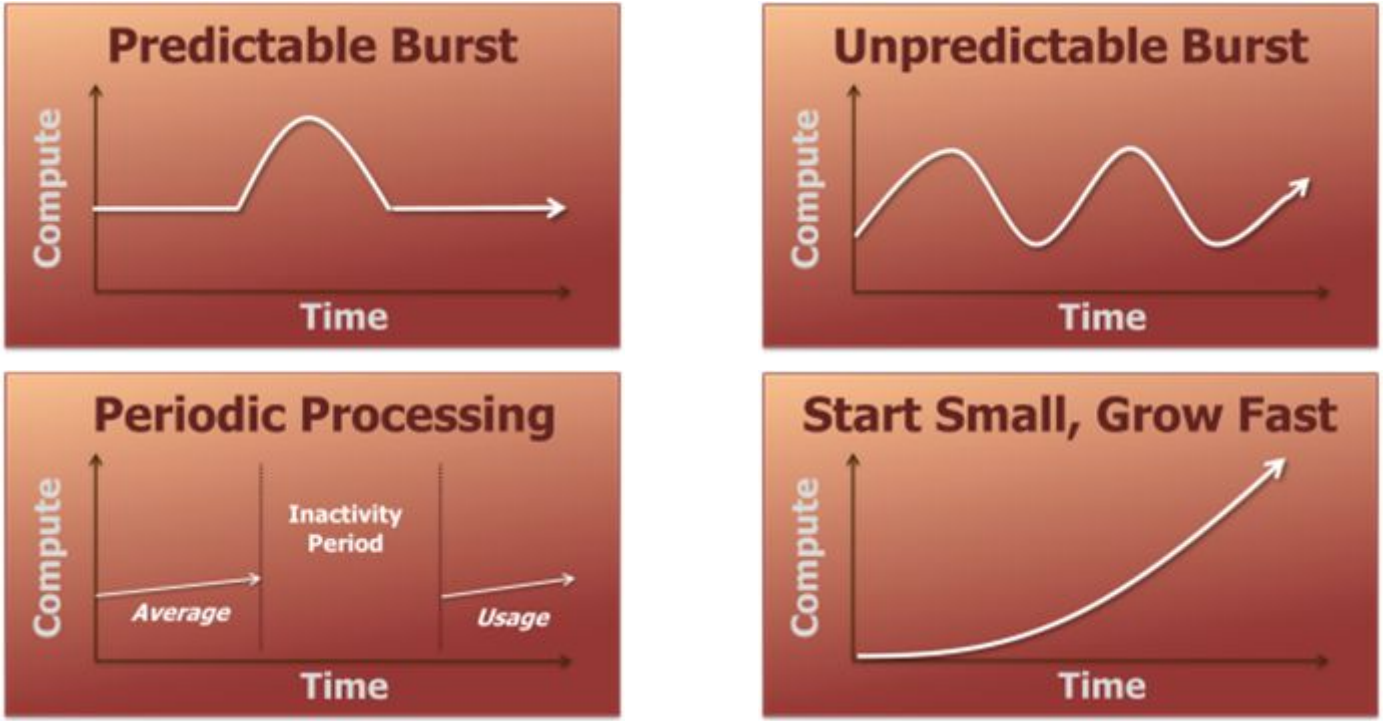
El objetivo de escalabilidad es el crecimiento,

- ***Respecto del tamaño:*** Agregando usuarios o recursos a controlar.
- ***Respecto de la distribución geográfica:*** Permitiendo dispersión.
- ***Respecto de los objetivos administrativos del sistema:*** Nuevas sintaxis, semánticas y servicios ofrecidos.

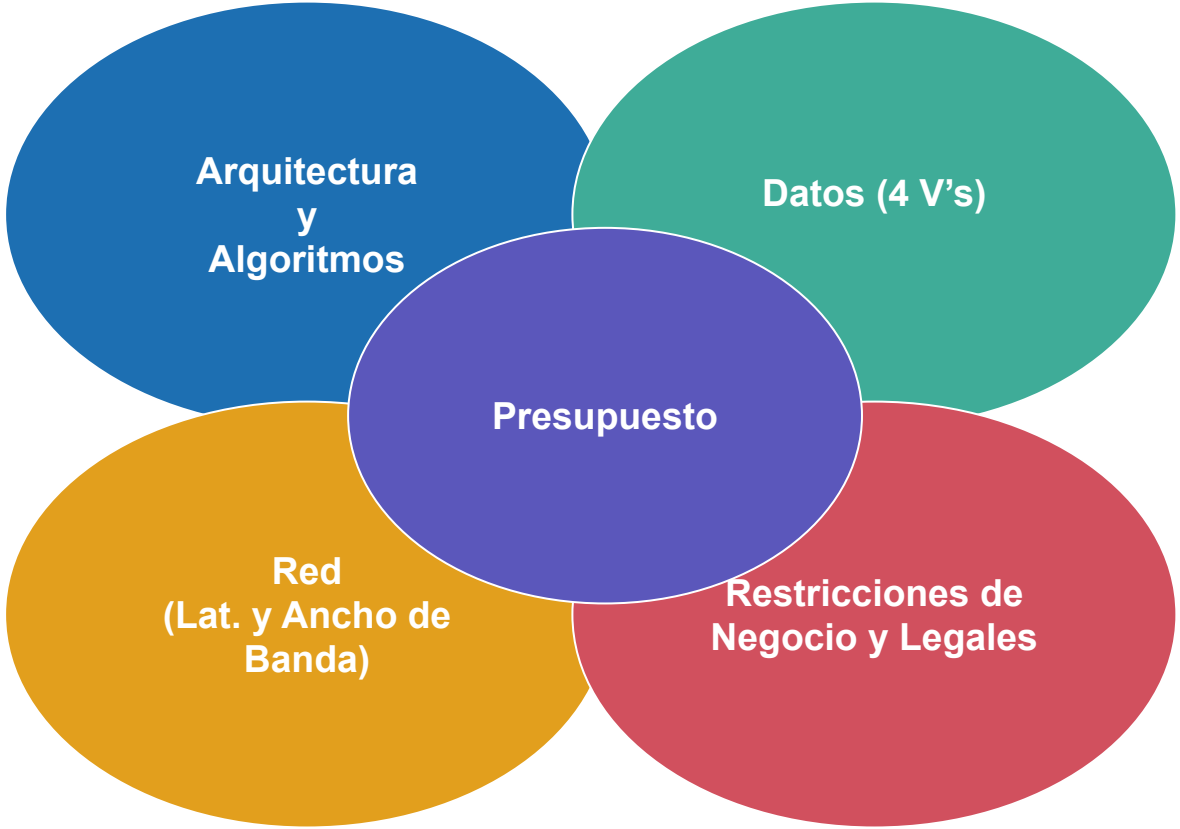
Escalabilidad | Características de las Plataformas

- Plataformas para alta concurrencia
  - Aplican patrones ya conocidos y probados
  - Escalamiento automático (con ciertos límites)
  - Vinculación fuerte con una infraestructura o producto
- Arquitecturas Ad-Hoc y Personalizadas
  - Necesidad de configuración y soporte
  - Escalamiento manual o automatizado por humanos
  - Posibilidad de migraciones a distintas plataformas

Escalabilidad | Patrón de Carga de Aplicaciones Web



Escalabilidad | Limitantes



Escalabilidad | Técnicas

- **Escalamiento vertical**
  - Agregar recursos a un nodo
- **Escalamiento horizontal**
  - Redundancia
  - Balanceadores de carga
  - Proximidad geografica
- **Fragmentación de datos**
  - ¿Qué datos deben estar juntos?
- **Componentización**
  - Separar servicios
- **Optimizar algoritmos**
  - Performance
  - Mensajería
- **Asincronismo**
  - ¿Qué debe ser necesariamente sincronico?
  - Limitado por el negocio

Agenda

- Escalabilidad
- **Elasticidad**
- Alta Disponibilidad

Elasticidad | Introducción

- **Escalabilidad**
  - Capacidad de un sistema para poder adaptarse a diferentes ambientes (*environments*) modificando los recursos del sistema
  - Término utilizado en muchos contextos
- **Elasticidad**
  - Capacidad de un sistema para poder modificar dinámicamente los recursos del sistema adaptándose a patrones de carga
  - Término utilizado en Arquitecturas Cloud

Elasticidad | Componentes

- ***Application Elastic Load Balancers***
  - Instancias nuevas reciben tráfico
  - Instancias dadas de baja dejan de recibir tráfico
  - *Endpoint* HTTP para detectar instancias nuevas
- ***Autoscaler***
  - *Scale In/Scale Out* en función de las métricas recolectadas
  - *Scale Out:* Incrementa instancias
  - *Scale In:* Decrementar instancias
- **Monitoring Automático**
  - Métricas sobre CPU, memoria, I/O, *networking*, etc. por cada instancia
  - Automatizado

Agenda

- Escalabilidad
- Elasticidad
- **Alta Disponibilidad**

Alta Disponibilidad | Introducción

Aún cuando ciertas propiedades de confiabilidad fallen para sistemas públicos, nos interesa que siempre estén disponibles:

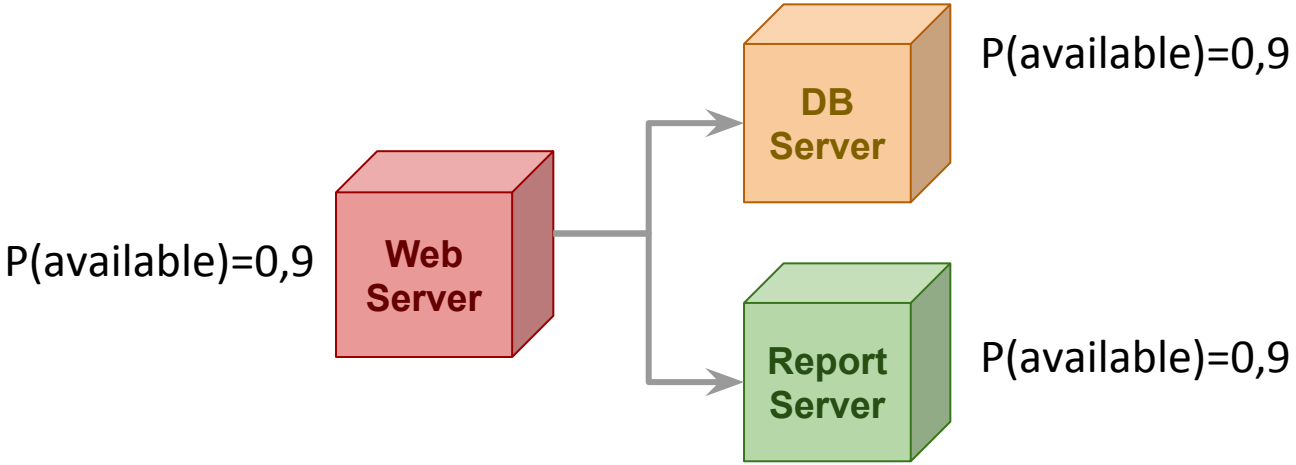
P(*system available*) = 1- P(*failure*)

Como **P(*system available*) < 1** siempre, lo importante es definir cuán cerca de 1 se encuentra:

- P(*system available*) ~ 0,9 => 36,5 días caídos al año
- P(*system available*) ~ 0,999 => 8,76 horas caídas al año

La disponibilidad de un sistema se mide por la cantidad de 9's

Escenario: Sistema distribuido con un componente por nodo

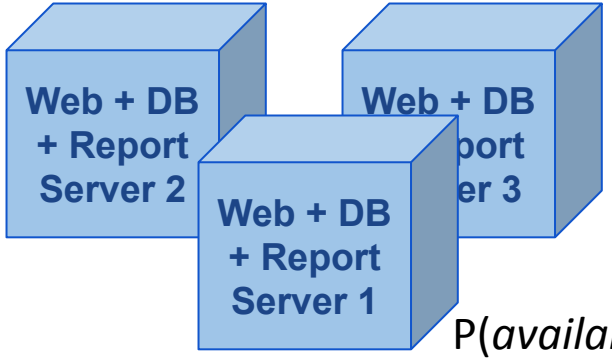


P(*available*) = P(*webAvailable*) . P(*dbAvailable*) . P(*reportAvailable*)

= 0,9 . 0,9 . 0,9 = **0,729**

Escenario: Sistema con redundancia con todos los componentes en c/nodo

P(*available 2*)=0,9



P(*available 3*)=0,9

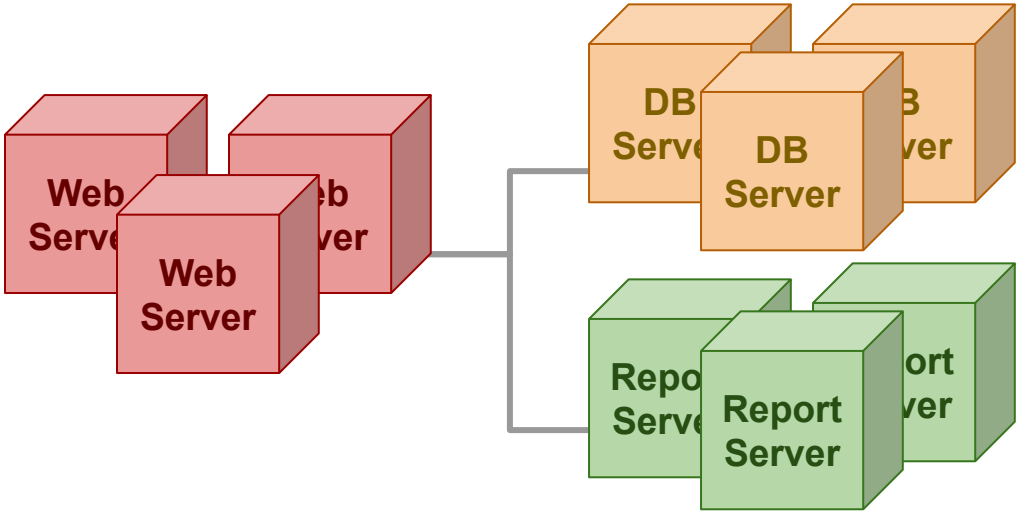
P(*available 1*)=0,9

P(*available*) = 1 - P(*failure*) = 1 - P(*failure 1 & failure 2 & failure 3*)

= 1 - 0,1 . 0,1 . 0,1 = 1 - 0,001 = **0,999**

Escenario: Sistema distribuido con clusters para cada componente

P(*cluster web*)=0,999



P(*cluster DB*)=0,999

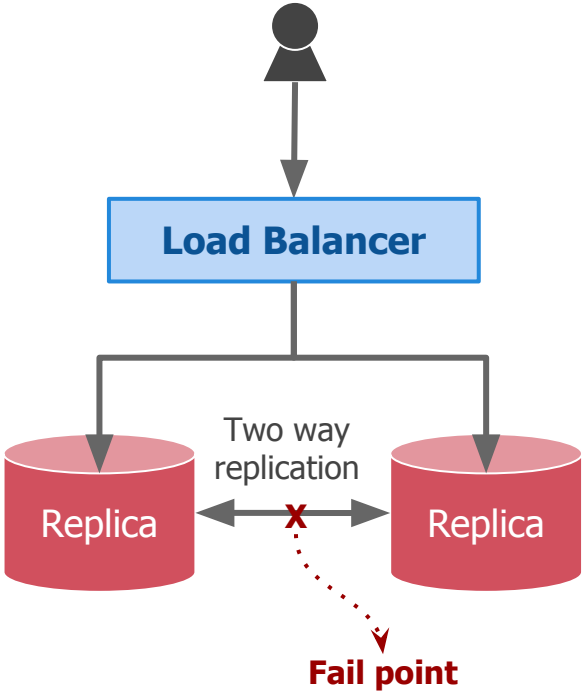
P(*cluster report*)=0,999

P(*available*) = P(*cluster web*) . P(*cluster DB*) . P(*cluster report*)
= 0,999 . 0,999 . 0,999 = **0,997**

CAP *Theorem* | Problemas frente a particionamiento

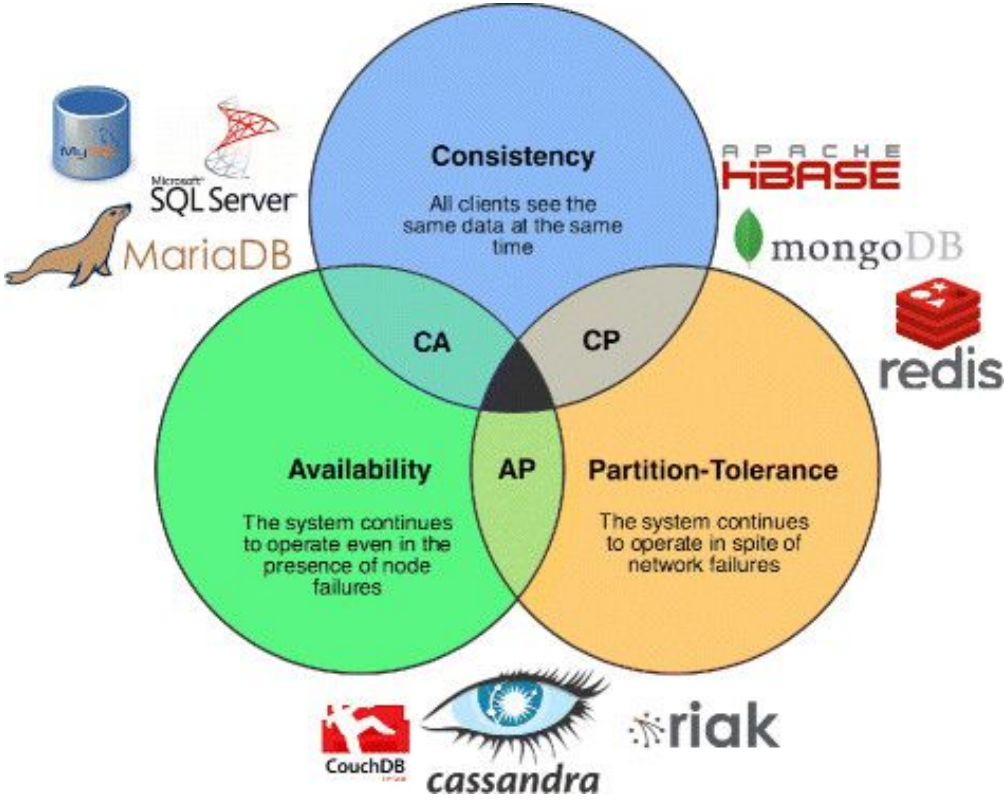
Eric Brewer ('98): en sistemas distribuidos con almacenamiento es posible garantizar sólo 2 de los siguientes atributos:

- ***Consistency:*** también conocido como repetibilidad de respuesta de todos los nodos frente a un mismo pedido.
- ***Availability:*** capacidad del sistema de responder a todo pedido.
- ***Partition Tolerance:*** capacidad de lidiar con la formación grupos aislados de nodos.



CAP *Theorem* | Ejemplos

- Se puede sacrificar consistency o availability. Pero no necesariamente es todo o nada.
- Sacrificar Partition-Tolerance significa no proveer un sistema distribuido.



Choosing the right NoSQL database for the job: a quality attribute evaluation, Lourenço et al. Journal of Big Data (2015)

Bibliografía

- G. Coulouris, J. Dollimore, t. Kindberg, G. Blair: Distributed Systems. Concepts and Design, 5th Edition, Addison Wesley, 2012.
  - Capítulo 18: Coordination and Agreement
- M. Van Steen, A. Tanenbaum: Distributed Systems. 3rd Edition. Pearson Education, 2017.
  - Capítulo 1.2: Introduction - Design Goals
  - Capítulo 8.1: Fault Tolerance - Introduction to Fault Tolerance